Real-Time Edge-Based Facial Recognition System Using Deep Learning on Raspberry Pi 4

Department of Electrical & Electronic Engineering
EEE 3202 – Capstone Project Design I

PROJECT OWNER: LABIB MARWAN HOQUE
ROLL: 2101064
SUPERVISOR: MD. MAYENUL ISLAM, ASSISTANT PROFESSOR, EEE, RUET

INTRODUCTION

- Context: Biometric security and smart access control are increasingly moving toward Edge Computing — processing data locally on the device rather than the cloud.).
- Proposed Solution: Implementation of Quantized Deep Learning Models (YuNet & SFace) via OpenCV's DNN module to achieve a robust, real-time recognition system running entirely offline.

OBJECTIVE

- Develop a standalone facial recognition system on Raspberry Pi 4 (Bookworm OS).
- Integrate lightweight Deep Neural Networks (YuNet for detection, SFace for recognition).
- Optimize for edge hardware to achieve usable real-time performance (3–4 FPS).
- Implement robust "Unknown" handling to detect and track unregistered individuals.

METHODOLOGY

The system pipeline consists of five stages:

1. **Input:** 5MP Pi Camera captures video via Picamera2 in RGB888 format.
2. **Preprocessing:** Frame mirroring (for user experience) and resizing to 320x320.
3. **Detection:** YuNet (Quantized CNN) locates faces and 5 key landmarks.
4. **Recognition:** SFace extracts a 128-D embedding vector from the aligned face.
5. **Matching:** Cosine Similarity compares the live vector against the stored database (encodings.pickle).

SYSTEM ARCHITECTURE

Hardware:

- Raspberry Pi 4 Model B (4GB): Central processing unit.
- Pi Camera V1.3 (5MP): Video acquisition.
- Cooling: Active cooling to maintain CPU clock speeds during inference.

Software Stack:

- Python 3.11: Core logic.
- OpenCV 4.6.0 (DNN): Deep Learning inference engine.
- Picamera2: Low-latency camera driver.
- NumPy: Vector mathematics.

TRAINING & DATABASE

To ensure ease of use and rapid deployment:

- **One-Shot Encoding:** The add_face.py script captures 15 variations of a user's face and instantly computes embeddings.
- **Storage:** Embeddings are serialized into a lightweight .pickle file, removing the need for model re-training.
- **Efficiency:** The main system loads the database in milliseconds upon startup.

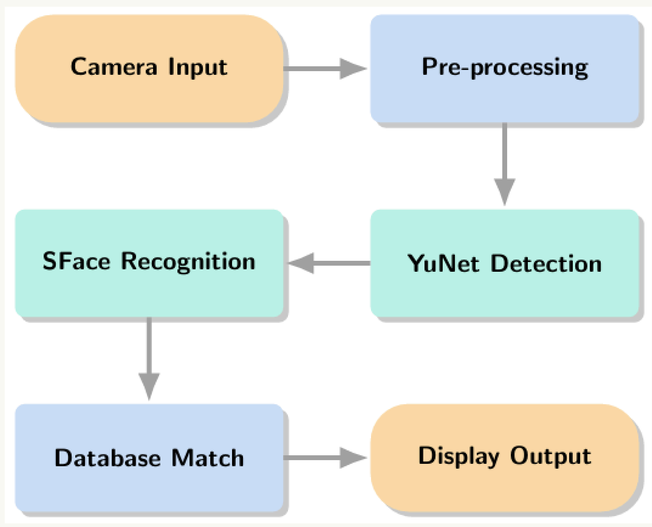

Figure 1: Overview of the project

RESULTS & ANALYSIS

The system was tested under various conditions to validate performance.

A. **Real-Time Performance**

- **Frame Rate:** Stable 3 – 4 FPS.
- **Latency:** < 300ms inference time per frame.
- **Analysis:** While lower than desktop GPUs, 4 FPS is sufficient for access control (e.g., unlocking a door as a person approaches).

B. **Accuracy & Robustness**

- **Confidence Threshold:** Set to 0.363 (Cosine) to minimize false positives.
- **Capabilities:**
  - Side Profiles: Detects faces up to ±30 degrees yaw.
  - Multi-Face: Tracks registered and unregistered users simultaneously.
  - Lighting: Consistent detection in low-light environments.

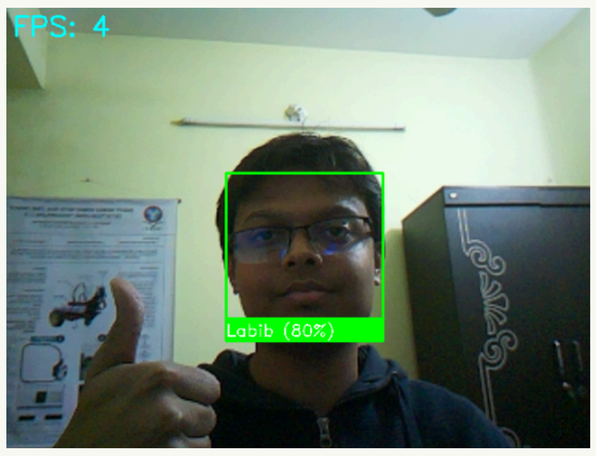

Figure 2: Camera interface (display output)

CONCLUSION

This work demonstrates reliable, offline facial recognition on a Raspberry Pi 4. Leveraging optimized YuNet and SFace models eliminates cloud dependency while ensuring high accuracy and usable real-time performance.

FUTURE SCOPE

- **Hardware Acceleration:** Integration with Google Coral USB Accelerator to boost FPS to 20+.
- **Database Integration:** Linking to SQL for time-stamped attendance logging.
- **Liveness Detection:** preventing spoofing attacks using 2D photographs.

BIBLIOGRAPHY

[1] F. Zhong, J. Deng, G. Yang, J. Chen, Y. Wei, and H. Zhang, "SFace: Sigmoid-Constrained Hypersphere Loss for Robust Face Recognition," IEEE Transactions on Image Processing, vol. 30, pp. 3687–3698, 2021.

[2] W. Wu, Y. Peng, and S. Yu, "YuNet: A Tiny Face Detector for Edge Devices," OpenCV Model Zoo, 2022. [Online]. Available: https://github.com/opencv/opencv_zoo.

[3] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, vol. 25, pp. 120–123, 2000.

[4] Raspberry Pi Foundation, "Raspberry Pi 4 Model B Datasheet," 2019. [Online]. Available: https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf.

[5] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in Proceedings of the 3rd International Conference on Learning Representations (ICLR), 2015.